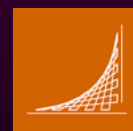




DOSW COMPANY



ESCUELA COLOMBIANA DE INGENIERIA

SEMANA 8

DOSW 1

EN QUE QUEDAMOS LA CLASE PASADA?

Maven™

JaCoCo
Java Code Coverage

sonarqube

JUnit

+



Project Lombok

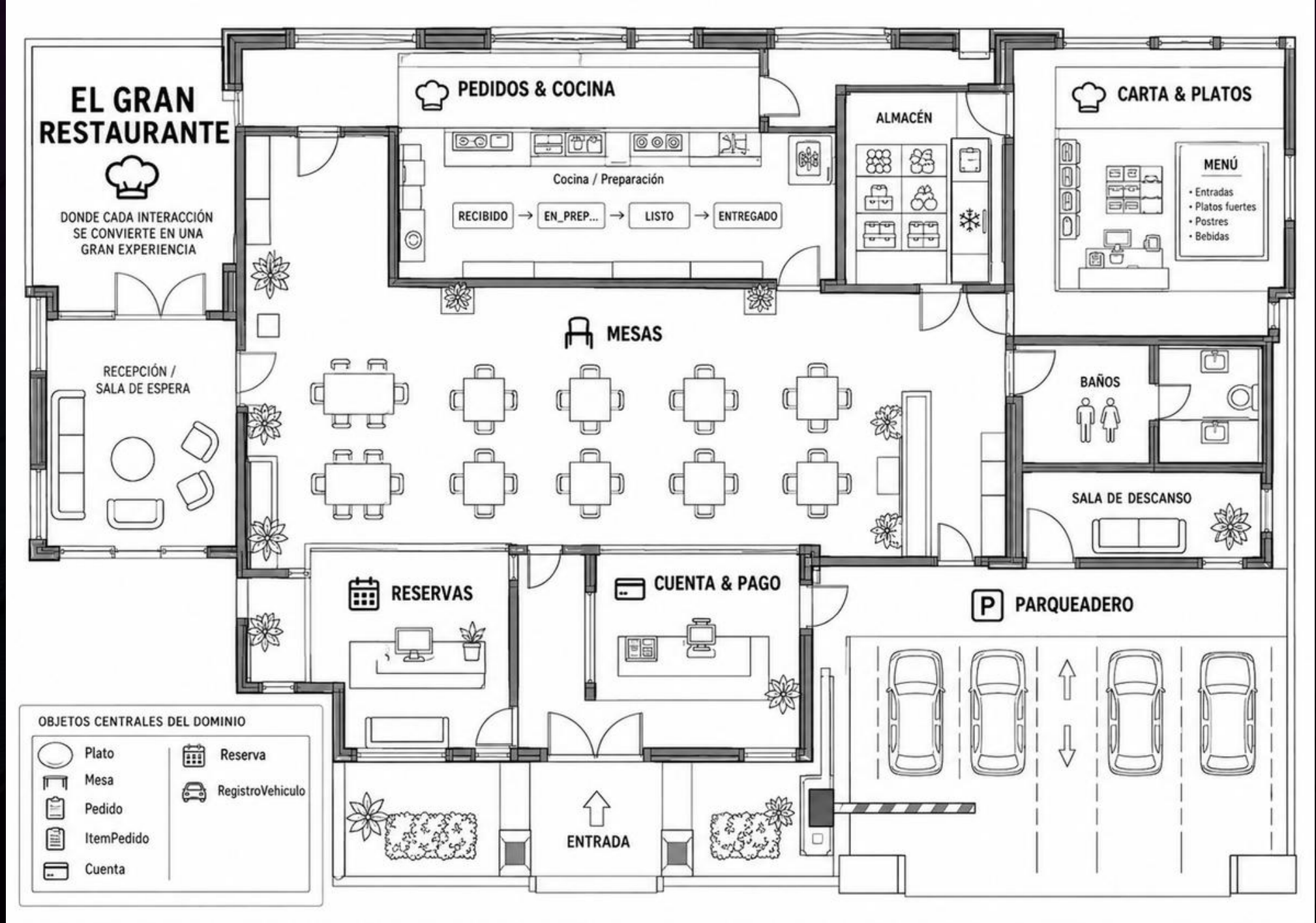
mapstruct

```

SRC/MAIN/JAVA
├── com.restaurante
│   ├── RestauranteApplication.java @SpringBoot
│   ├── controller/
│   │   ├── PlatoController.java Controller
│   │   ├── GlobalExceptionHandler.java Controller
│   ├── service/
│   │   ├── PlatoService.java Service
│   │   ├── PlatoServiceImpl.java Service
│   ├── repository/
│   │   ├── PlatoRepository.java Repository
│   ├── model/
│   │   ├── domain/
│   │   │   ├── Plato.java Dominio
│   │   ├── entity/
│   │   ├── dto/
│   │   │   ├── request/
│   │   │   │   ├── PlatoRequestDTO.java Controller
│   │   │   ├── response/
│   │   │   │   ├── PlatoResponseDTO.java Controller
│   ├── mapper/
│   │   ├── PlatoMapper.java Dominio
│   │   ├── PlatoEntityMapper.java Dominio
│   ├── validator/
│   │   ├── PlatoValidator.java Service
│   ├── exception/
│   │   ├── PlatoNotFoundException.java Dominio
│   │   ├── PlatoAlreadyExistsException.java Dominio
│   ├── config/
│   │   ├── SwaggerConfig.java Config
│   │   ├── CorsConfig.java Config

```

=





TU RESTAURANTE A PUNTO DE FUNCIONAR

Vamos a iniciar a construir nuestra aplicacion, la idea es revisar los objetos centrales del dominio, cada funcionalidad del sistema, validaciones que podemos tener y sobretodo para el dia de hoy

VAMOS A REVISAR COMO FUNCIONA

El manejo de errores, documentar nuestra aplicacion, ponerle visibilidad a nuestros flujos y aprender a diagramar



EL SISTEMA COMPLETO - DOMINIOS Y OBJETOS



ANTES DE ESCRIBIR CODIGO

Es necesario ver como se relacionan los objetos dentro de nuestro sistema (el core)
El diagrama de clases se vuelve tu mapa de navegacion para implementar posteriormente

LOS DOMINIOS QUE ACABAMOS DE VER

EL SISTEMA COMPLETO - DOMINIOS Y OBJETOS



Plato

ID Atributos principales

- ID id
- T nombre
- \$ precio
- 📁 categoría
- ✅ disponible

⚙️ Métodos principales

- 🔄 esValido()
- 🕒 cambiarDisponibilidad()



Mesa

ID Atributos principales

- ID id
- T numero
- \$ capacidad
- 👤 estado
- ✉️ cuentaAbierta

⚙️ Métodos principales

- 🔄 estaDisponible()
- 🔑 abrirCuenta()
- 🕒 cerrarCuenta()



Pedido

ID Atributos principales

- ID id
- 🪑 mesa
- 📄 items
- 📅 estado
- ✅ timestamp

⚙️ Métodos principales

- 🕒 puedeModificarse()
- 🔑 agregarItem()
- 🕒 cambiarEstado()



ItemPedido

ID Atributos principales

- ID id
- T idPlato
- \$ nombre
- 📅 precioCongelado
- 🔑 cantidad

⚙️ Métodos principales

- 🕒 calcularSubtotal()
- 🕒 actualizarCantidad()
- 📄 esValido()

EL SISTEMA COMPLETO - DOMINIOS Y OBJETOS



Cuenta

ID Atributos principales

ID idMesa

T total

\$ estado

📅 fechaApertura

⚙ Métodos principales

🔢 calcularTotal()

🕒 agregarCargo()

🔑 cerrarCuenta()



Reserva

ID Atributos principales

ID id

T mesa

👤 cliente

🕒 fechaHora

👥 comensales

⚙ Métodos principales

🕒 estaVigente()

🕒 cancelar()

🔄 reprogramar()



RegistroVehiculo

ID Atributos principales

ID placa

👤 entrada

\$ salida

🕒 estado

✅ cobro

⚙ Métodos principales

🔢 calcularCobro()

📅 registrarSalida()

🕒 estaActivo()



EL SISTEMA COMPLETO - DOMINIOS Y OBJETOS

Hay relaciones que debes tener claras

Pedido con ItemPedido : Un pedido tiene muchos items, pero cada item congela el precio del plato en el momento del pedido — no sigue el precio actual.

Mesa con Cuenta : Una mesa puede tener a lo sumo una cuenta abierta a la vez. No puedes crear un nuevo pedido en una mesa sin cuenta.

RegistroVehiculo: Es independiente del resto del dominio — no se relaciona con mesas ni pedidos.

COMO SERIA PASAR NUESTRO DIAGRAMA DE CLASES A CODIGO?



En la guia de estudio habian ciertas partes que faltaban?
Lo pudieron identificar?

Para estas clases y Enums como son las relaciones entonces?

AHORA VAMOS A CODIFICAR

Transforma el diagrama de clases en
clases a nivel de código

Utiliza Lombok para generar sus métodos
más rápido (Getters, setters,
constructores, etc)

Guarda todas las clases en
model/domain





DOSW COMPANY

YA INICIAMOS A CONSTRUIR PERO.... NECESITAMOS ALGO



Un API sin estándares es como un restaurante donde cada mesero hace las cosas a su manera. Hoy definimos las reglas que todos deben seguir.



ESTANDARES DE UN API REST



ESTANDARES DE UN API REST

✓ Garantizar la coherencia entre aplicaciones ya que Independiente de quien desarrolle o en donde se desarrolle existe una coherencia de principio a fin

✓ Cualquier API debe basarse en el protocolo HTTP utilizando sus metodos adecuadamente

✓ Las rutas deben ser claras y consistentes – sustantivos en plural y jerarquias logicas

✓ La respuesta debe ser consistente en su estructura, puede incluir paginacion, enlaces, totales, etc.

✓ La api debe responser con los codigos de estado adecuados – por verbo Http





ESTANDARES DE UN API REST



Las respuestas de error y de happy path deben ser claras con un mensaje descriptivo y un código (error o aprobación) interno



La documentacion debe ser clara y accesible, siendo actualizada y navegable para los usuarios – Swagger/OpenAPI genera la documentación automáticamente desde el código. Si el código cambia, la doc cambia y no hay un esfuerzo manual.



La comunicacion debe ser intuitiva, confiable y consistente





CODIGOS HTTP POR VERBO

Cada verbo tiene sus propios códigos de éxito y sus propios errores esperados. Aprenderlos te evita devolver un **200** cuando creaste algo, o un **500** cuando simplemente no encontraste el recurso.

GET

Solo lee — nunca modifica estado. Seguro e idempotente.

SITUACIÓN	CÓDIGO	CUÁNDO USARLO	EJEMPLO EN EL RESTAURANTE
Éxito con datos	200 OK	Siempre que devuelva datos	GET /platos → lista de platos
Recurso no encontrado	404 Not Found	El id no existe	GET /platos/999 → plato no existe
Parámetro inválido	400 Bad Request	Query param con formato incorrecto	GET /platos?precio=abc → precio no es número
Sin permisos	403 Forbidden	El rol no tiene acceso a ese recurso	GET /reportes (cliente sin rol admin)



CODIGOS HTTP POR VERBO

POST

Crea un nuevo recurso. No idempotente — llamarlo N veces crea N recursos.

SITUACIÓN	CÓDIGO	CUÁNDO USARLO	EJEMPLO EN EL RESTAURANTE
✓ Recurso creado	201 Created	Siempre que crees algo nuevo exitosamente	POST /platos → plato creado con id asignado
✗ Datos de entrada inválidos	400 Bad Request	@Valid falló — campos vacíos, formato incorrecto	POST /platos con nombre vacío o precio negativo
✗ Ya existe	409 Conflict	Violación de unicidad de negocio	POST /platos con nombre "Tacos" que ya existe
✗ Regla de negocio violada	422 Unprocessable	Formato OK pero la regla de dominio falló	POST /pedidos con plato agotado o mesa sin cuenta abierta



CODIGOS HTTP POR VERBO

PUT

Reemplaza el recurso completo. Idempotente — el mismo resultado N veces.

SITUACIÓN	CÓDIGO	CUÁNDO USARLO	EJEMPLO EN EL RESTAURANTE
✓ Reemplazado correctamente	200 OK	Recurso actualizado, devuelves el objeto actualizado	PUT /platos/5 → plato actualizado completo
✓ Creado porque no existía	201 Created	PUT puede crear si no existe (upsert) — no siempre se implementa así	PUT /platos/99 — no existe, se crea
✗ Recurso no encontrado	404 Not Found	El id del path no existe	PUT /platos/999 → plato no existe
✗ Body incompleto	400 Bad Request	PUT exige todos los campos — falta alguno	PUT /platos/5 sin precio en el body

PATCH

Actualiza parcialmente. Solo los campos que envías cambian.

SITUACIÓN	CÓDIGO	CUÁNDO USARLO	EJEMPLO EN EL RESTAURANTE
✓ Actualizado parcialmente	200 OK	Devuelves el objeto completo ya actualizado	PATCH /platos/5/disponible → solo cambia disponible=false
✗ Recurso no encontrado	404 Not Found	El id del path no existe	PATCH /pedidos/99/estado → pedido no existe
✗ Transición de estado inválida	422 Unprocessable	El cambio viola una regla de negocio de estado	PATCH /pedidos/5/estado con CANCELADO cuando ya está LISTO
✗ Valor inválido en el body	400 Bad Request	El valor enviado no es un estado válido del enum	PATCH /pedidos/5/estado con "COCINADO" (no existe ese estado)



CODIGOS HTTP POR VERBO

DELETE

Elimina el recurso. Idempotente — borrarlo N veces tiene el mismo resultado.

SITUACIÓN	CÓDIGO	CUÁNDO USARLO	EJEMPLO EN EL RESTAURANTE
✓ Eliminado correctamente	204 No Content	Siempre que borres algo — sin body de respuesta	DELETE /platos/5 → eliminado, response body vacío
✗ Recurso no encontrado	404 Not Found	El id no existe	DELETE /platos/999 → plato no existe
✗ No se puede eliminar	409 Conflict	Regla de negocio impide el borrado	DELETE /platos/5 si ese plato tiene pedidos activos
✗ Estado no permite eliminar	422 Unprocessable	La entidad está en un estado que bloquea la eliminación	DELETE /pedidos/5 cuando está EN_PREPARACION (ya lo tomó cocina)

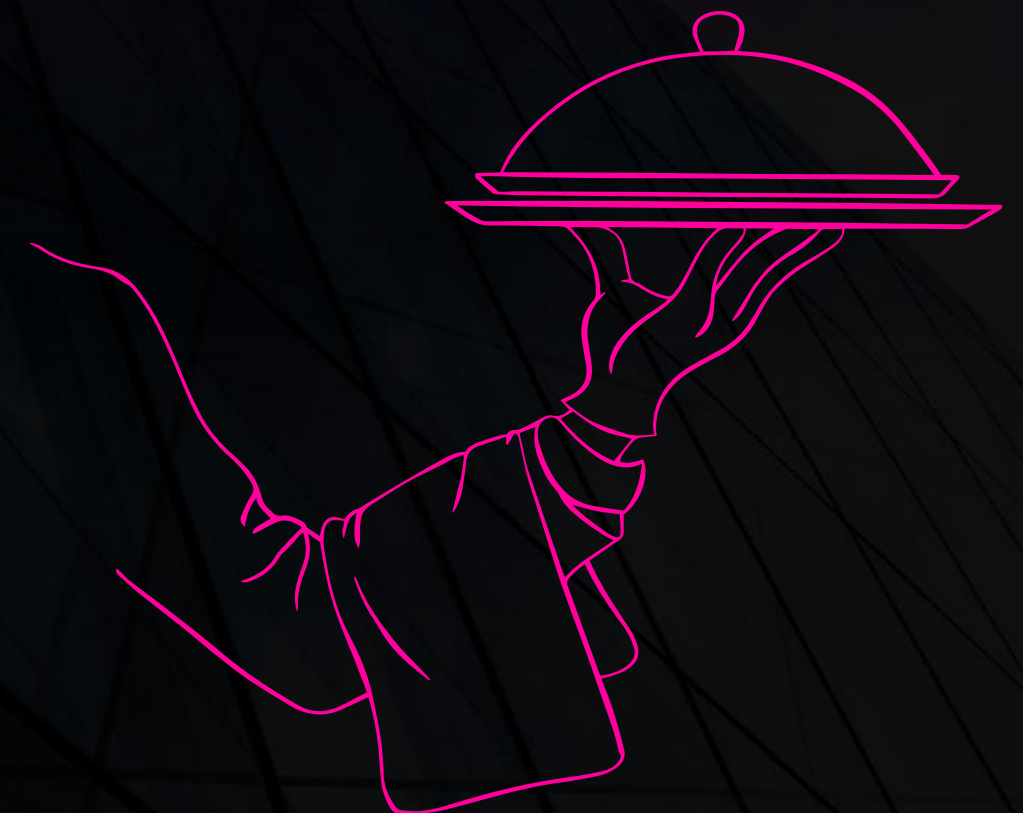
IMPORTANTE

Hay algunos codigos que se confunden como el 200 y el 201 – 200 es todo bien aqui estan los datos, 201 es cree algo nuevo
404 vs 422 – 404 es el recurso no existe, 422 es el recurso existe pero la operacion viola alguna regla de negocio
400 vs 422 – 400 es el formato de lo enviado esta mal, 422 es el formato esta bien pero el negocio lo rechaza



ES MOMENTO DE CREAR NUESTRAS FUNCIONALIDADES

Un error común es empezar a crear clases sin saber qué van a hacer. El primer paso es listar todas las funcionalidades del sistema y agruparlas por dominio — eso te dice cuántos controllers, servicios y cuáles pueden compartirse.





FUNCIONALIDAD	DESCRIPCIÓN
 Gestión de Reservas	Crear una reserva para una mesa, actualizar datos, cancelar, consultar por usuario. También contempla entrar sin reserva (walk-in).
 Gestión del Menú	Ver la carta completa, filtrar por categoría, marcar platos como disponibles o agotados, ver descripción y precio.
 Gestión de Platos	Crear un plato, editar nombre/precio/descripción, eliminar de la carta. Solo el administrador.
 Gestión de Órdenes/Pedidos	Confirmar un pedido con lista de platos y mesa, cambiar estado (cocina), ver pedidos activos, cancelar si está en RECIBIDO.
 Gestión de Parqueadero	Registrar entrada de vehículo, registrar salida, consultar disponibilidad de cupos, listar vehículos activos.
 Reportes del Día	Plato más pedido, ingresos por categoría, total de pedidos, mesas con cuenta abierta. Solo con streams (sin BD).



RECUERDAN LOS DOMINIOS QUE DEFINIMOS?

Cada grupo de endpoints aquí corresponde a un Controller en tu proyecto. Un Controller = un dominio. Así de limpia debe ser la estructura.

PlatoController /api/v1/platos — La carta del restaurante				
ENDPOINT	VERBO	✓ ÉXITO	✗ ERRORES ESPERADOS	
/api/v1/platos	GET	200	403 sin permisos	
/api/v1/platos/{id}	GET	200	404 no existe	
/api/v1/platos	POST	201	400 inválido · 409 nombre duplicado	
/api/v1/platos/{id}	PUT	200	400 body incompleto · 404 no existe	
/api/v1/platos/{id}/disponible	PATCH	200	404 no existe	
/api/v1/platos/{id}	DELETE	204 sin body	404 no existe · 409 tiene pedidos activos	



PedidoController /api/v1/pedidos — El flujo de cocina

ENDPOINT	VERBO	✓ ÉXITO	✗ ERRORES ESPERADOS
/api/v1/pedidos	POST	201	400 inválido · 422 plato agotado
/api/v1/pedidos	GET	200	403 sin permisos
/api/v1/pedidos/{id}	GET	200	404 no existe
/api/v1/pedidos/{id}/estado	PATCH	200	404 no existe · 422 transición inválida
/api/v1/pedidos/{id}	DELETE	204 sin body	404 no existe · 422 ya en cocina

ReservaController /api/v1/reservas — Gestión de reservas

ENDPOINT	VERBO	✓ ÉXITO	✗ ERRORES ESPERADOS
/api/v1/reservas	POST	201	400 inválido · 409 mesa ya reservada ese horario
/api/v1/reservas	GET	200	403 sin permisos
/api/v1/reservas/{id}	GET	200	404 no existe
/api/v1/reservas/{id}	PUT	200	404 no existe · 409 conflicto de horario
/api/v1/reservas/{id}	DELETE	204 sin body	404 no existe · 422 reserva ya completada



ParqueaderoController /api/v1/parqueadero — Acceso de vehículos

ENDPOINT	VERBO	✓ ÉXITO	✗ ERRORES ESPERADOS
/api/v1/parqueadero/entrada	POST	201	409 placa ya registrada · 422 parqueadero lleno
/api/v1/parqueadero/salida/{placa}	POST	200	404 placa no activa
/api/v1/parqueadero/activos	GET	200	403 sin permisos

ReporteController /api/v1/reportes — Inteligencia del negocio

ENDPOINT	VERBO	✓ ÉXITO	✗ ERRORES ESPERADOS
/api/v1/reportes/resumen	GET	200	403 sin permisos
/api/v1/reportes/platos-populares	GET	200	403 sin permisos
/api/v1/reportes/ingresos	GET	200	403 sin permisos · 400 rango de fechas inválido



FALTAN DOMINIOS NO?



MESAS

El salón del restaurante

- Ver estado de todas las mesas
- Abrir / cerrar cuenta de mesa
- Capacidad y disponibilidad



CUENTA & PAGO

El cierre del ciclo

- Ver cuenta abierta de una mesa
- Precio congelado al pedir
- Registrar pago y cerrar cuenta
- Una cuenta por mesa a la vez

Como los construirian? Cuales serian los endpoints y que salida esperarian?

Menu es = a

Plato?

La diferencia es semántica: el menú es lo que ve el cliente, los platos son lo que administra el gerente. Mismo servicio, distintos endpoints, distintas rutas, distintas validaciones. – Como lo arreglarian?



CREA TUS CONTROLADORES

Agrega cada controlador en la carpeta controller – RECUERDA UTILIZAR spring boot para la generacion de cada uno y sus endpoints (funcionalidades)

PERO VAMOS A INICIAR CON LO SENCILLO - PLATO Y MENU

Analiza cada funcionalidad Para identificar sus DTO (request y response)

Requiere de datos de entrada y de salida?

Hay datos que son opcionales?

Que deberia responder en caso de exito o de error?

CREA LOS DTOS INICIALES QUE IDENTIFIQUES



CUALES ENTONCES SERIAN LOS SERVICIOS?

Que necesitamos?

Que podemos reutilizar?

LA CLAVE DE LA REUTILIZACIÓN

Si un PedidoService necesita saber si un plato existe y está disponible antes de confirmarlo. En vez de duplicar esa lógica, **inyecta IPlatoService dentro de PedidoServiceImpl**. Un servicio puede depender de otro servicio — siempre via interfaz, nunca via implementación concreta.

Con eso respondo Necesito datos de otro dominio?

Si MenuController y PlatoController ambos listan platos → el mismo método `IPlatoService.obtenerDisponibles()` sirve a los dos. No duplicar lógica — ampliar la interfaz.

Con eso respondo Existe un metodo que hace esto?



CREA LOS SERVICIOS

Agrega cada Servicio en la carpeta service – recuerda los metodos utilizan como puntos de entrada (si necesitan) son los objetos de dominio

NO LOS DTOS

Analiza cada funcionalidad Para identificar con nuestras clases de dominio si el o los servicios para Menu y Plato los necesitan

Recuerda que tienes que crear el contrato del servicio (Interfaz) y su implementacion

Para la implementacion usa un Array en memoria donde cada metodo utilizara streams para operar sobre el

**PERO COMO SE HABLA
ENTONCES EL CONTROLADOR Y
EL SERVICIO SI LOS OBJETOS
NO SON IGUALES?**



DOSW COMPANY

STREAMS - LISTAS SIN BD

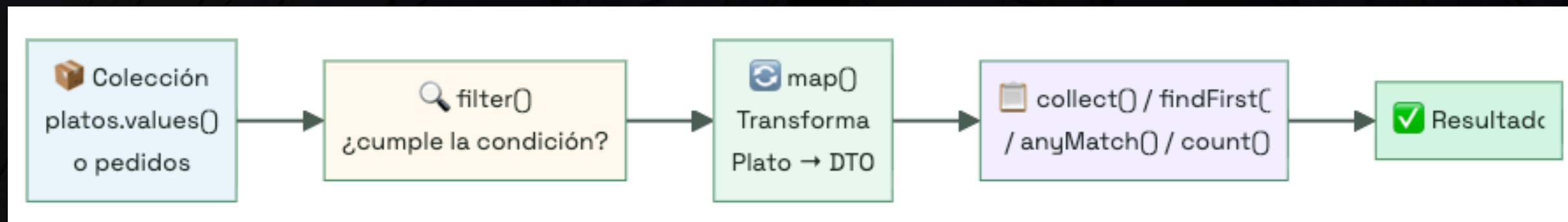
Sin base de datos todavía, todo vive en listas y mapas en memoria. Los Streams son la forma elegante de consultarlos — y seguirán siendo útiles cuando llegue la BD.





STREAMS - LISTAS SIN BD

Sin base de datos aún, los servicios usan `Map<Long, Plato>` y `List<Pedido>` en memoria. Los Streams son la forma de consultar, filtrar y transformar esas colecciones de forma elegante — y también los usaremos cuando tengamos BD.





DOSW COMPANY

OPERACIONES DEL SERVICE USANDO STREAMS

```
// — Obtener toda la carta (Map → List de DTOs) —————  
@Override  
public List<PlatoResponseDTO> obtenerTodos() {  
    return platos.values().stream()           // Stream de todos los Plato  
        .map(mapperOut::toDTO)               // transforma cada Plato a DTO  
        .collect(Collectors.toList());       // colecta a lista  
}
```

```
// — Filtrar solo los disponibles —————  
@Override  
public List<PlatoResponseDTO> obtenerDisponibles() {  
    return platos.values().stream()  
        .filter(Plato::isDisponible)         // solo los disponibles  
        .map(mapperOut::toDTO)  
        .toList();                           // Java 16+: .toList() en vez de collect(toList())  
}
```



DOSW COMPANY

OPERACIONES DEL SERVICE USANDO STREAMS

```
// — En PedidoService: resumen del día con streams —————
public ResumenDiaDTO resumenDelDia() {

    long totalPedidos = pedidos.stream().count();

    double ingresoTotal = pedidos.stream()
        .flatMap(p → p.getItems().stream())          // aplana la lista de items de todos los pedidos
        .mapToDouble(item → item.getPrecio() * item.getCantidad())
        .sum();

    Map<String, Long> platosMasPedidos = pedidos.stream()
        .flatMap(p → p.getItems().stream())
        .collect(Collectors.groupingBy(
            item → item.getNombrePlato(),
            Collectors.counting()
        ));

    return new ResumenDiaDTO(totalPedidos, ingresoTotal, platosMasPedidos);
}
```



ES HORA DE LOS MAPPERS

Crea los mappers en la carpeta correspondiente

RECUERDA UTILIZAR MAPSTRUCT

Los Request se transformaran en el objeto de dominio correspondiente
No te preocupes si no todos los campos del objeto de dominio se llenan inicialmente

EN EL PROCESAMIENTO DE LA INFORMACION ESA ESTRUCTURA SE COMPLEMENTARA

Los mappers tienen sus divisiones in y out

Recuerda son interfaces





UNA VEZ IMPLEMENTADOS...

Es momento de inyectar las dependencias

EJEMPLO:

```
// Ejemplo: PedidoServiceImpl reutiliza IPLatoService
@Service
@Slf4j
@RequiredArgsConstructor
public class PedidoServiceImpl implements IPedidoService {

    private final IPLatoService    platoService;    // ← reutiliza otro Service via interfaz
    private final PedidoMapperIn    mapperIn;
    private final PedidoMapperOut    mapperOut;

    private final List<Pedido> pedidos = new ArrayList<>(); // en memoria por ahora
```

Esto tambien pasa en los controladores, donde inyectamos nuestros servicios y nuestros mappers





CORRE LA APLICACION



Ejecuta tu aplicacion con springboot

Revisa que todo este inyectado correctamente

Cuidado con dependencias circulares: si A depende de B y B de A, Spring no puede resolver la inyección.

Una vez lo ejecutes podras ver tu aplicacion en el localhost:8080

Utiliza postman para poder operar sobre lo que hemos creado hasta el momento





Al probar con POSTMAN necesitamos tener claro nuestros objetos de entrada y de salida

Que pasa si hago un cambio en mi API

Necesito modificar constantemente mi coleccion de POSTMAN

OPENAPI GENERA LA DOCUMENTACIÓN DIRECTAMENTE DESDE EL CÓDIGO. SI EL CÓDIGO CAMBIA, LA DOC CAMBIA. NUNCA MÁS POSTMAN DESACTUALIZADO.

SWAGGER UI



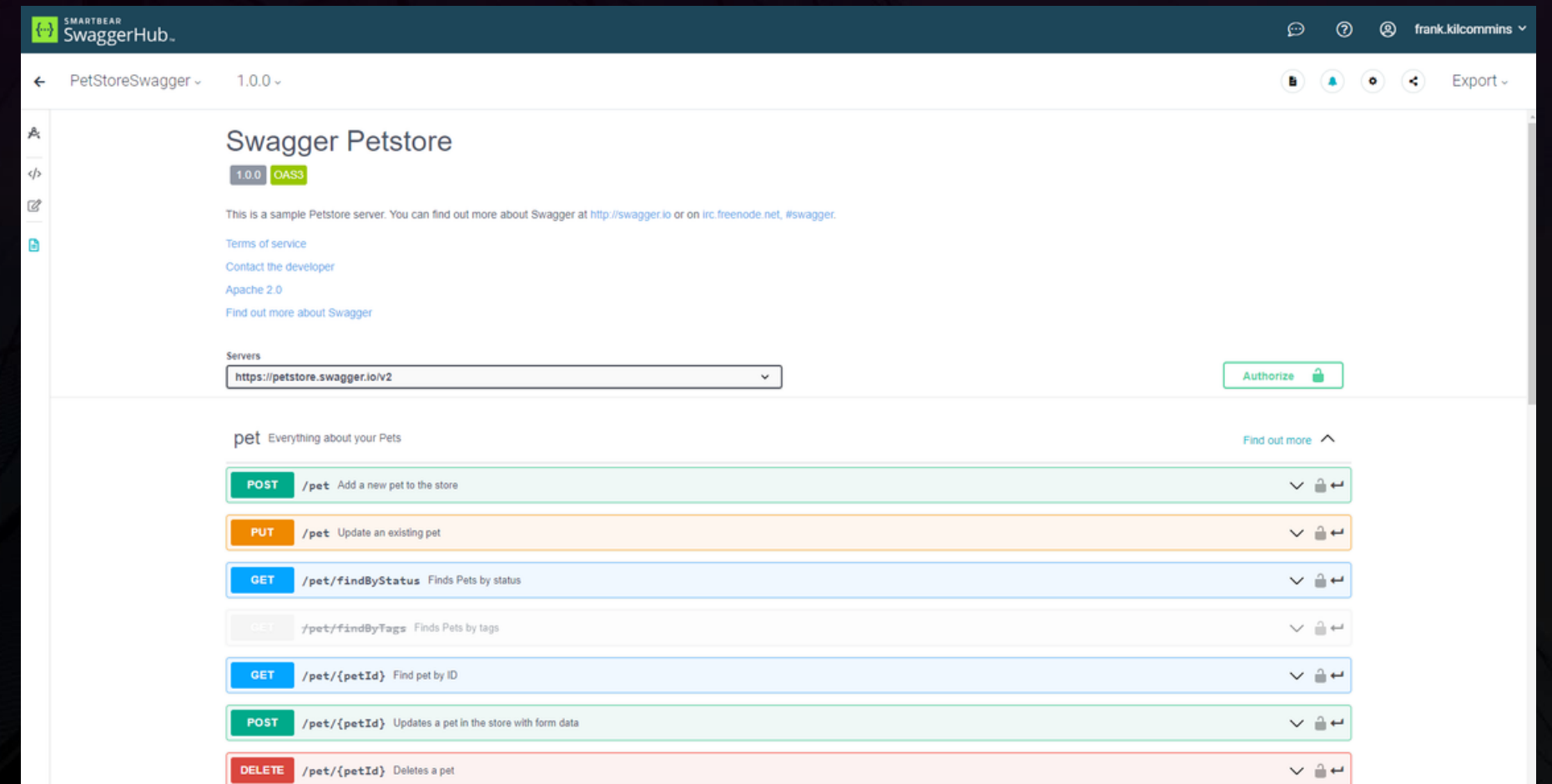
SWAGGER UI

Es una interfaz web que genera automaticamente una documentacion interactiva de la API Rest a partir de una especificacion OPEN API

- Documentación viva que se actualiza sola
 - Se puede probar desde el navegador
- Frontend ve exactamente qué entra y qué sale
 - Contrato estandarizado (OpenAPI spec)

Sin Swagger

- Postman con colecciones desactualizadas
- Frontend pregunta "¿qué campos acepta?"
- Documentación en Word que nadie actualiza
- QA no puede probar sin el backend corriendo





PORQUE ES IMPORTANTE?



Permite tener una documentacion clara y siempre actualizada



Facilita mucho las pruebas funcionales del API, ya que se pueden ejecutar sin necesidad de usar POSTMAN o CURL



Mejora la colaboracion ya que Backend, Frontend y QA entienden facilmente la API



Basado en la especificacion OPEN API, permite estandarizar.



COMO SE IMPLEMENTA? - IMPLEMENTACIÓN EN TU RESTAURANTE

1. **Agregar dependencia en pom.xml:** springdoc-openapi genera Swagger automáticamente desde las anotaciones de Spring.

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

2. Acceder a Swagger UI — ya está disponible sin más config: Con la dependencia ya agregada la documentacion esta lista automaticamente, por defecto estara disponible en:

- Documentación en JSON → <http://localhost:8080/v3/api-docs>
- Swagger UI (interfaz gráfica) → <http://localhost:8080/swagger-ui.html>

<http://localhost:8080/swagger-ui/index.html>



COMO SE IMPLEMENTA? - IMPLEMENTACIÓN EN TU RESTAURANTE

4. Documentar los endpoints con anotaciones `@Operation` y `@ApiResponse`
Le dice a Swagger qué hace cada endpoint y qué respuestas puede dar.

```
import org.springframework.web.bind.annotation.*;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;

@RestController
@RequestMapping("/users")
@Tag(name = "Usuarios", description = "Operaciones relacionadas con usuarios")
public class UserController {

    @GetMapping("/{id}")
    @Operation(summary = "Obtener usuario por ID", description = "Devuelve un usuario b")
    public User getUserById(@PathVariable Long id) {
        return new User(id, "Andres Cantor", "andres@example.com");
    }

    @PostMapping
    @Operation(summary = "Crear un nuevo usuario", description = "Registra un usuario e")
    public User createUser(@RequestBody User user) {
        user.setId(100L);
        return user;
    }
}
```

```
@RestController
@RequestMapping("/api/v1/platos")
@Tag(name = "Platos", description = "Gestión de la carta del restaurante")
public class PlatoController {

    @Operation(
        summary = "Obtener todos los platos",
        description = "Devuelve la carta completa con estado de disponibilidad"
    )
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Carta obtenida correctamente"),
        @ApiResponse(responseCode = "500", description = "Error interno del servidor")
    })
    @GetMapping
    public ResponseEntity<List<PlatoResponseDTO>> obtenerTodos() {
        return ResponseEntity.ok(platoService.obtenerTodos());
    }

    @Operation(summary = "Crear un plato", description = "Agrega un nuevo plato a la carta")
    @ApiResponses({
        @ApiResponse(responseCode = "201", description = "Plato creado correctamente"),
        @ApiResponse(responseCode = "400", description = "Datos de entrada inválidos"),
        @ApiResponse(responseCode = "409", description = "Ya existe un plato con ese nombre")
    })
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public PlatoResponseDTO crear(@RequestBody @Valid PlatoRequestDTO dto) {
        return platoService.crear(dto);
    }
}
```



COMO SE IMPLEMENTA? - IMPLEMENTACIÓN EN TU RESTAURANTE

5. Prueba en tu navegador, arranca la aplicación con `mvn spring-boot:run` o ejecutando directamente tu clase `main` y abre tu navegador

OPENAPI VS SWAGGER

OpenAPI es el estándar (el "contrato" en YAML/JSON que describe tu API). Swagger UI es la herramienta que lee ese contrato y lo muestra de forma interactiva en el navegador. `springdoc-openapi` genera el contrato automáticamente desde tu código Spring y levanta la UI.



NOS FUNCIONO A TODOS?



- Levanta el proyecto con `mvn spring-boot:run`
- Abre: `http://localhost:8080/swagger-ui/index.html`
- ¿Ves los endpoints de tu restaurante listados? → ☒ Funcionó
- ¿La página no carga? → Verifica que agregaste la dependencia en el `pom.xml`
- ¿Los endpoints no aparecen? → Verifica que el `@RestController` esté en el paquete correcto



MANEJO DE LOGS EN UN API REST

Sin logs no sabes qué pasó, cuándo pasó ni quién lo causó. Con @Slf4j de Lombok, tener logs cuesta una anotación.



LOGIN

El logging es el registro estructurado de eventos que ocurren en la aplicación (ejecuciones normales, errores, advertencias, etc.).

PERMITEN

Detectar y diagnosticar errores

Auditar operaciones sensibles

Medir el comportamiento en tiempos de respuesta o uso de endpoints

Facilita un atributo de calidad: Observabilidad

UN RESTAURANTE SIN REGISTROS NO SABE QUÉ SALIÓ MAL, QUIÉN PIDIÓ QUÉ, NI EN QUÉ MOMENTO COLAPSÓ LA COCINA. EL LOGGING ES LO MISMO EN SOFTWARE: EL REGISTRO ESTRUCTURADO DE TODO LO QUE LE PASA A LA APLICACIÓN.



LIBRERIAS COMUNES

SLF4J : API DE LOGGING

LOGBACK: Implementacion por defecto en Spring Boot

Log4j2: Otra implementacion bastante usada

NIVELES DE LOGS

NIVEL	CUÁNDO USARLO	EJEMPLO EN EL RESTAURANTE
TRACE	Detalle extremo — solo en desarrollo activo	"Entrando al mapper, campo nombre = 'Tacos'"
DEBUG	Depuración — info útil al desarrollar	"Buscando plato con id=5 en la lista"
INFO	Eventos normales — flujo exitoso	"Pedido #42 creado para mesa 3"
WARN	Algo extraño pero no roto	"Plato 'Salmón' marcado como agotado — horario de cierre"
ERROR	Fallo que necesita atención	"No se pudo procesar el pedido #42 — plato no encontrado"



EJEMPLO DE USO

```
// En cualquier clase del proyecto - con Lombok es un @Slf4j
@Service
@Slf4j                                // ← Lombok genera: private static final Logger log = LoggerFactory.getLogger(...)
@RequiredArgsConstructor
public class PlatoServiceImpl implements IPlatoService {

    private final Map<Long, Plato> platos = new HashMap<>();
    private final PlatoMapperIn    mapperIn;
    private final PlatoMapperOut   mapperOut;

    @Override
    public PlatoResponseDTO crear(PlatoRequestDTO dto) {
        log.info("Creando plato con nombre '{}'", dto.nombre());           // INFO - evento normal

        boolean nombreDuplicado = platos.values().stream()
            .anyMatch(p → p.getNombre().equalsIgnoreCase(dto.nombre()));

        if (nombreDuplicado) {
            log.warn("Intento de crear plato duplicado: '{}'", dto.nombre()); // WARN - no es error pero es sospechoso
            throw new PlatoYaExisteException("Ya existe un plato con ese nombre: " + dto.nombre());
        }

        Plato plato = mapperIn.toDomain(dto);
        plato.setId(generarId());
        platos.put(plato.getId(), plato);

        log.info("Plato '{}' creado con id={}", plato.getNombre(), plato.getId()); // INFO - éxito
        return mapperOut.toDTO(plato);
    }
}
```

```
@Override
public PlatoResponseDTO buscarPorId(Long id) {
    log.debug("Buscando plato con id={}", id);                               // DEBUG - detalle de búsqueda
    Plato plato = platos.get(id);
    if (plato == null) {
        log.error("Plato con id={} no encontrado", id);                       // ERROR - fallo que hay que manejar
        throw new PlatoNoEncontradoException("Plato no encontrado con id: " + id);
    }
    return mapperOut.toDTO(plato);
}
```



EJEMPLO DE USO

Puedes controlar los niveles de logging:

properties

```
# application.properties
logging.level.root=INFO
logging.level.com.ejemplo.api=DEBUG
logging.file.name=logs/api.log
```

O en `application.yml`:

yaml

```
logging:
  level:
    root: INFO
    com.ejemplo.api: DEBUG
  file:
    name: logs/api.log
```

Root es el nivel base para todo

Si estableces un paquete en especifico puedes dar mas detalle

IMPORTANTE

@Slf4j de Lombok genera automáticamente la variable log en la clase. Sin Lombok tendrías que escribir `private static final Logger log = LoggerFactory.getLogger(PlatoServiceImpl.class);` en cada clase. Con Lombok solo pones @Slf4j arriba de la clase.



PAUSA RAPIDA - COMPRUEBA TU CODIGO

¿Corrió el proyecto con @Slf4j? ¿Aparece algo en la consola al hacer una petición?

Si el log.info aparece en la consola → funciona. Si no aparece, revisa el nivel configurado en application.yml.

¿Qué nivel usas para el flujo exitoso de negocio?

INFO — es el nivel para eventos normales del sistema.
DEBUG para detalles de desarrollo, WARN para situaciones extrañas, ERROR para fallos que requieren atención.



MANEJO DE CONFIGURACIONES



APPLICATION PROPERTIES VS YML

En Spring boot los archivos de configuracion son el punto central para definir propiedades del sistema: como lo son los puertos, la conexion con base de datos, la configuracion de los logs y variables personalizadas

spring boot permite dos formatos: .properties y .yml

```
properties

# Puerto de la API
server.port=8081

# Configuración de base de datos
spring.datasource.url=jdbc:mysql://localhost:3306/miapp
spring.datasource.username=root
spring.datasource.password=secret

# Logs
logging.level.root=INFO
logging.level.com.miapp=DEBUG

# Propiedades personalizadas
miapp.api.key=123456789
```

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/miapp
    username: root
    password: secret

logging:
  level:
    root: INFO
    com.miapp: DEBUG

miapp:
  api:
    key: 123456789
```



APPLICATION PROPERTIES VS YML

Ejemplo de archivos:

- application-dev.yml
- application-test.yml
- application-prod.yml

Archivo principal (application.yml):

```
yaml  
  
spring:  
  profiles:  
    active: dev
```

Ejemplo de application-dev.yml:

```
yaml  
  
server:  
  port: 8080  
spring:  
  datasource:  
    url: jdbc:mysql://localhost:3306/devdb
```

Ejemplo de application-prod.yml:

```
yaml  
  
server:  
  port: 9090  
spring:  
  datasource:  
    url: jdbc:mysql://prod-db:3306/proddb
```

Permite definir perfiles por tipo de ambiente y así separar sus configuraciones

Se pueden usar desde el código con anotaciones como `@Value` o generando un Configuration properties utilizando la anotación `@ConfigurationProperties`

Nunca expongan contraseñas o secretos en los repositorios

Usar .yml si la configuración es compleja, .properties si es simple



**YA TENEMOS
DOCUMENTACION, YA
TENEMOS LOGS QUE NOS
FALTA??**



MANEJO DE EXCEPCIONES



VALIDACIONES DE INPUT VS VALIDACIONES DE NEGOCIO

Hay dos tipos de validación y cada una vive en un lugar diferente. Mezclarlas es uno de los errores más comunes en el primer proyecto.

La regla de oro

Las validaciones de input van en el DTO (con anotaciones Bean Validation). Las validaciones de negocio van en el Service (con excepciones propias). Nunca al revés.

- ✓ Mensajes claros de error para el cliente.
- ✓ No confiar solo en la UI → Validar siempre en el backend.
- ✓ Combinar ambas: primero input, luego negocio.
- ✓ Centralizar reglas de negocio → No duplicarlas en varios servicios.

VALIDACIONES DE INPUT GARANTIZAN QUE LOS DATOS SON CORRECTOS EN FORMA.

VALIDACIONES DE NEGOCIO GARANTIZAN QUE LOS DATOS SON VÁLIDOS EN EL CONTEXTO DE LA APLICACIÓN.



MANEJO DE ERRORES EN UN SISTEMA

Validación de INPUT — en el DTO

- ✗ → ¿El formato es correcto?
- ✗ → ¿El campo está presente?
- ✗ → ¿El número está en rango?
- ✗ → @NotNull, @NotBlank, @Min, @Max
- ✗ → Spring la dispara automáticamente con @Valid
- ✗ → Da **400** Bad Request

Validación de NEGOCIO — en el Service

- ✓ ¿Cumple la regla del dominio?
- ✓ ¿El plato ya existe con ese nombre?
- ✓ ¿El pedido está en estado modificable?
- ✓ Excepciones propias tipadas
- ✓ Tú la dispararas manualmente con throw
- ✓ Da **409** Conflict o **422** Unprocessable



COMO SERIA EN NUESTRO RESTAURANTE?

Bean validation en el DTO del restaurante

```
public record PlatoRequestDTO(  
  
    @NotBlank(message = "El nombre del plato es obligatorio")  
    @Size(min = 2, max = 80, message = "El nombre debe tener entre 2 y 80 caracteres")  
    String nombre,  
  
    @NotNull(message = "El precio es obligatorio")  
    @DecimalMin(value = "0.01", message = "El precio debe ser mayor a cero")  
    Double precio,  
  
    @NotBlank(message = "La categoría es obligatoria")  
    @Pattern(  
        regexp = "ENTRADA|PLATO_FUERTE|POSTRE|BEBIDA",  
        message = "Categoría inválida. Opciones: ENTRADA, PLATO_FUERTE, POSTRE, BEBIDA"  
    )  
    String categoria,  
  
    @Size(max = 500, message = "La descripción no puede superar 500 caracteres")  
    String descripcion // ← no es @NotBlank, puede ser null/vacío  
  
) {}
```

Activa la validacion en el controller con @Valid

```
@PostMapping  
@ResponseStatus(HttpStatus.CREATED)  
public PlatoResponseDTO crear(  
    @RequestBody @Valid PlatoRequestDTO dto) { // ← @Valid activa Bean Validation  
    return platoService.crear(dto);  
    // Si el DTO tiene campos inválidos, Spring lanza MethodArgumentNotValidException  
    // ANTES de que llegue al service – el GlobalExceptionHandler lo captura  
}
```



COMO SERIA EN NUESTRO RESTAURANTE?

Validaciones de negocio propias en la logica de tu restaurante – Puedes mandarlas a un validator para simplificar la logica y que tenga unica responsabilidad

```
// — Regla de negocio #1: nombres únicos en la carta —————
boolean nombreDuplicado = platos.values().stream()
    .anyMatch(p → p.getNombre().equalsIgnoreCase(dto.nombre()));
if (nombreDuplicado) {
    throw new PlatoYaExisteException("Ya existe un plato: " + dto.nombre());
}

// — Regla de negocio #2 (ejemplo parrilla): corte requiere término —
// if (dto.categoria().equals("PARRILLA") && dto.terminoCoccion() == null) {
//     throw new TerminoCoccionRequeridoException("Los cortes de parrilla requieren término de cocción");
// }

// — Regla de negocio #3 (ejemplo japonés): máx 6 rolls por tanda —
// long rollsActivos = platos.values().stream()
//     .filter(p → p.getCategoria().equals("ROLL") && p.isDisponible()).count();
// if (rollsActivos ≥ 6) {
//     throw new CapacidadExcedidaException("Solo pueden prepararse 6 rolls por tanda");
// }
```

```
// — Regla de negocio: solo se puede modificar en estado RECIBIDO —
Pedido pedido = pedidos.stream()
    .filter(p → p.getId().equals(id))
    .findFirst()
    .orElseThrow(() → new PedidoNoEncontradoException("Pedido no encontrado: " + id));

if (pedido.getEstado() ≠ EstadoPedido.RECIBIDO && nuevoEstado == EstadoPedido.CANCELADO) {
    throw new EstadoInvalidoException(
        "Un pedido en estado " + pedido.getEstado() + " no puede cancelarse"
    );
}
```

MANEJO DE ERRORES EN UN SISTEMA

- ERRORES PERSONALIZADOS –
Propagados
- TRY – CATCH – Para capturar excepciones y saber como tratarlas
- ERROR HANDLER – El gerente que intercepta los problemas





ERROR HANDLER - EL GERENTE DEL RESTAURANTE

- Es una clase o componente que intercepta los errores que ocurren en la aplicación.
 - Permite no llenar el código de try/catch en todos los controladores.
 - Garantiza que el cliente reciba respuestas uniformes (códigos HTTP y mensajes claros).



EL `GLOBALEXCEPTIONHANDLER` ES EL GERENTE DEL RESTAURANTE. CUANDO EL MESERO TIENE UN PROBLEMA (ERROR EN EL CÓDIGO), EN VEZ DE SALIR CORRIENDO O DESMAYARSE FRENTE AL CLIENTE (500 SIN CONTEXTO), EL GERENTE INTERCEPTA EL PROBLEMA Y LE EXPLICA AL CLIENTE QUÉ PASÓ DE FORMA PROFESIONAL.



PASO 1 – LAS EXCEPCIONES PERSONALIZADAS DEL RESTAURANTE

```
public class PlatoNoEncontradoException extends RuntimeException {  
    public PlatoNoEncontradoException(String mensaje) { super(mensaje); }  
}  
  
public class PlatoYaExisteException extends RuntimeException {  
    public PlatoYaExisteException(String mensaje) { super(mensaje); }  
}  
  
public class EstadoInvalidoException extends RuntimeException {  
    public EstadoInvalidoException(String mensaje) { super(mensaje); }  
}  
  
public class PedidoNoModificableException extends RuntimeException {  
    public PedidoNoModificableException(String mensaje) { super(mensaje); }  
}  
  
// ← Cada concepto de restaurante puede tener las suyas:  
// TerminoCoccionRequeridoException (parrilla)  
// RollCapacidadExcedidaException (japonés)  
// PescaNoDisponibleException (comida de mar)
```



PASO 2 – EL ERRORRESPONSE DTO (RESPUESTA UNIFORME DE ERROR)

```
// dto/ErrorResponseDTO.java – formato estándar para todos los errores
public record ErrorResponseDTO(
    LocalDateTime timestamp,
    int            status,
    String         error,
    String         message,
    String         path
) {
    // Factory method para crear respuestas rápidamente
    public static ErrorResponseDTO of(int status, String error, String message, String path) {
        return new ErrorResponseDTO(LocalDateTime.now(), status, error, message, path);
    }
}
```



PASO 3 — EL GLOBALEXCEPTIONHANDLER

```
// exception/GlobalExceptionHandler.java
@RestControllerAdvice      // ← intercepta excepciones de TODOS los controllers
@Slf4j
public class GlobalExceptionHandler {

    // — Excepciones de input (Bean Validation) → 400 —————
    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ErrorResponseDTO handleValidacion(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {

        // Extrae TODOS los errores de validación en una sola lista
        String errores = ex.getBindingResult().getFieldErrors().stream()
            .map(e → e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.joining(" | "));

        log.warn("Validación de input fallida: {}", errores);

        return ErrorResponseDTO.of(400, "Datos de entrada inválidos", errores, request.getRequestURI());
    }
}
```

```
// — Recurso no encontrado → 404 —————
@ExceptionHandler(PlatoNoEncontradoException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public ErrorResponseDTO handleNoEncontrado(
    PlatoNoEncontradoException ex, HttpServletRequest request) {

    log.warn("Recurso no encontrado: {}", ex.getMessage());
    return ErrorResponseDTO.of(404, "No encontrado", ex.getMessage(), request.getRequestURI());
}

// — Conflicto de negocio → 409 —————
@ExceptionHandler(PlatoYaExisteException.class)
@ResponseStatus(HttpStatus.CONFLICT)
public ErrorResponseDTO handleConflicto(
    PlatoYaExisteException ex, HttpServletRequest request) {

    log.warn("Conflicto de negocio: {}", ex.getMessage());
    return ErrorResponseDTO.of(409, "Conflicto", ex.getMessage(), request.getRequestURI());
}
```



PASO 3 – EL GLOBALEXCEPTIONHANDLER

```
// — Estado inválido de negocio → 422 —————
@ExceptionHandler(EstadoInvalidoException.class)
@ResponseStatus(HttpStatus.UNPROCESSABLE_ENTITY)
public ErrorResponseDTO handleEstadoInvalido(
    EstadoInvalidoException ex, HttpServletRequest request) {

    log.warn("Estado de negocio inválido: {}", ex.getMessage());
    return ErrorResponseDTO.of(422, "Estado inválido", ex.getMessage(), request.getRequestURI());
}

// — Cualquier otro error no controlado → 500 —————
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public ErrorResponseDTO handleGeneral(Exception ex, HttpServletRequest request) {
    log.error("Error no controlado: {}", ex.getMessage(), ex); // ← full stack trace en ERROR
    return ErrorResponseDTO.of(500, "Error interno", "Algo salió mal, intenta de nuevo", request.getRequestURI());
    // NUNCA expongas ex.getMessage() aquí – puede filtrar info sensible
}
```

Si cada Controller maneja sus propios errores con try-catch, el formato de respuesta varía de endpoint a endpoint y duplicas código. Con un solo `@RestControllerAdvice` capturas todas las excepciones en un solo lugar y garantizas respuestas uniformes en toda la API.

A veces lo que podemos hacer es capturar una excepcion mapearla a nuestro objeto base y así transformarla en el DTO correspondiente



PASO 3 – EL GLOBALEXCEPTIONHANDLER

EXCEPCIÓN	CAUSA	STATUS CODE	MENSAJE DE EJEMPLO
<code>MethodArgumentNotValidException</code>	@Valid falló en el DTO	400	"nombre: no puede estar vacío"
<code>PlatoNoEncontradoException</code>	GET/PUT con id inexistente	404	"Plato no encontrado con id: 99"
<code>PlatoYaExisteException</code>	POST con nombre duplicado	409	"Ya existe un plato: Tacos"
<code>EstadoInvalidoException</code>	Cambio de estado inválido	422	"Un pedido EN_PREPARACION no puede cancelarse"
<code>Exception</code>	Cualquier error no previsto	500	"Algo salió mal, intenta de nuevo"

Hagamos una prueba en vivo

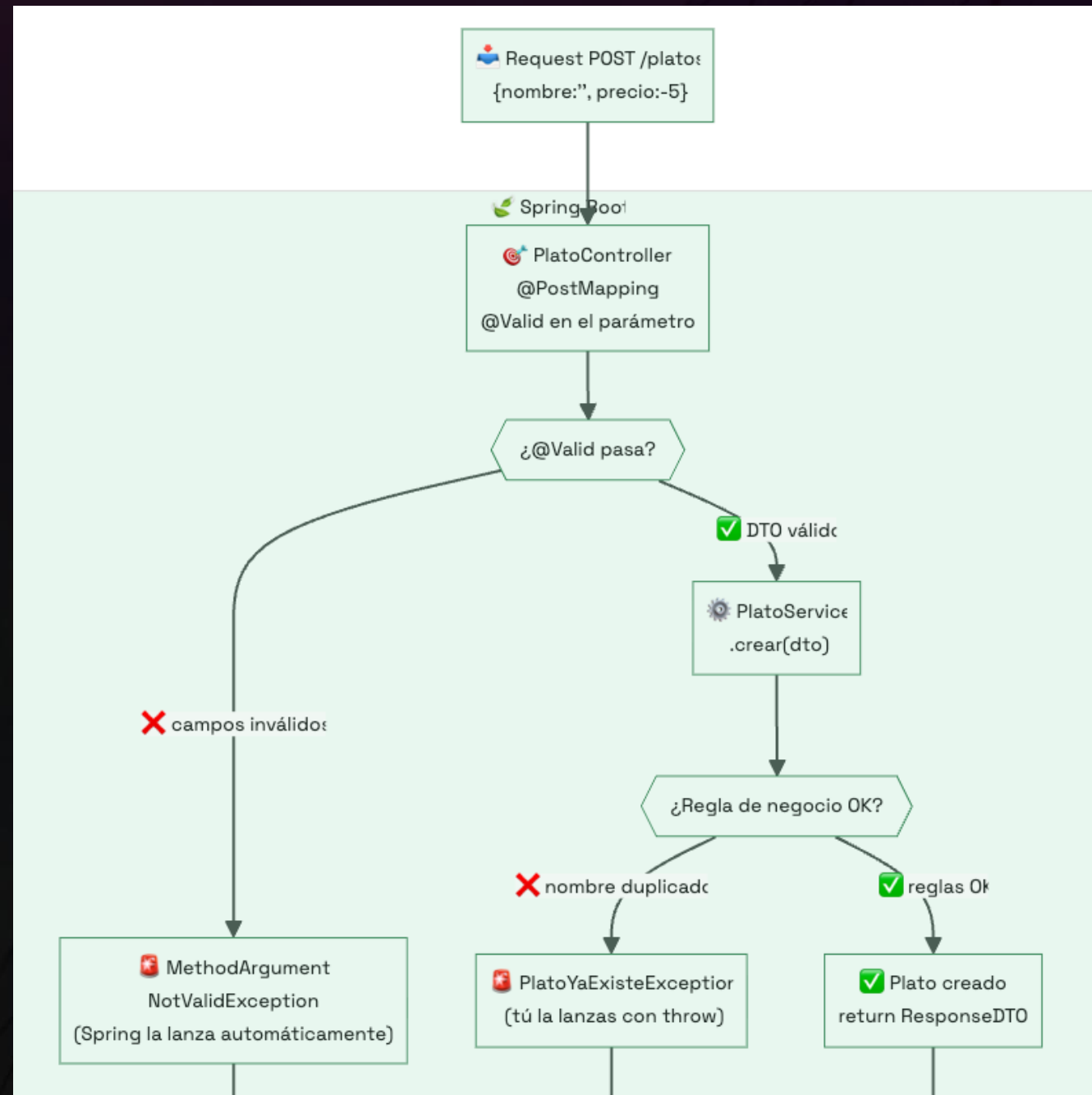
HAZ UN POST CON BODY VACÍO AL ENDPPOINT DE TU RESTAURANTE. ¿QUÉ RECIBES?

¿POR QUÉ EL `@ExceptionHandler` DE `Exception.class` VA AL FINAL?

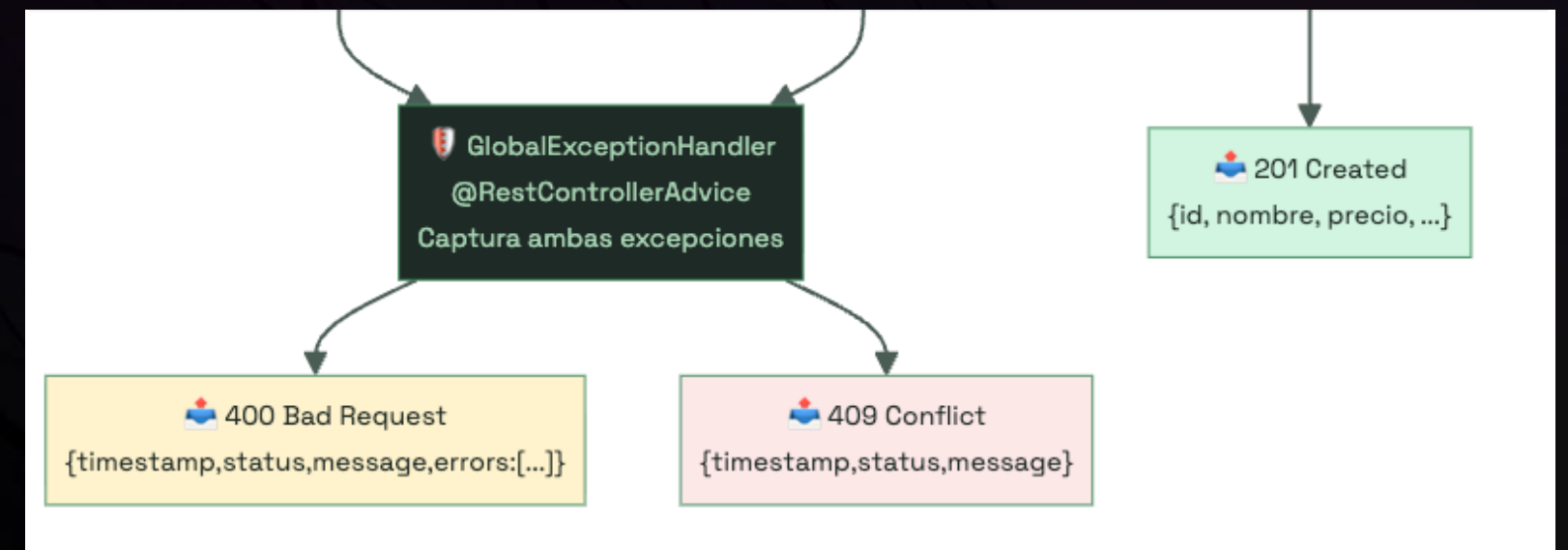
Spring evalúa los handlers de más específico a más general. Si el genérico fuera primero, capturaría todo antes de que los específicos puedan actuar.

DIAGRAMA DE VALIDACION

1



2





DOSW COMPANY

OPERACIONES DEL SERVICE USANDO STREAMS

```
// — Buscar por categoría —————  
@Override  
public List<PlatoResponseDTO> obtenerPorCategoria(String categoria) {  
    return platos.values().stream()  
        .filter(p → p.getCategoria().equalsIgnoreCase(categoria))  
        .map(mapperOut::toDTO)  
        .toList();  
}
```

```
// — Verificar si nombre ya existe (devuelve boolean) —————  
private boolean nombreExiste(String nombre) {  
    return platos.values().stream()  
        .anyMatch(p → p.getNombre().equalsIgnoreCase(nombre));  
}  
  
// — Buscar por id o lanzar excepción —————  
private Plato buscarOLanzar(Long id) {  
    return Optional.ofNullable(platos.get(id))  
        .orElseThrow(() → new PlatoNoEncontradoException("Plato no encontrado: " + id));  
}
```



VERSIONAMIENTO DE SERVICIOS Y APIS

El cliente que consume tu API no puede actualizarse el mismo día que tú despliegas. El versionamiento te da tiempo para migrar sin apagar nada.



VERSIONAMIENTO DE UNA API

- Imagina que el frontend ya está en producción usando `/api/v1/platos`. Ahora necesitas cambiar la estructura de la respuesta — agregar un campo, renombrar otro, eliminar uno que usaba el cliente. Si cambias el endpoint directamente, el frontend se rompe. La solución: versionar el API.

¿Cuándo versionar?

Cuando un cambio rompe la compatibilidad con los clientes existentes: eliminar un campo de la respuesta, cambiar el tipo de un campo, renombrar un campo obligatorio, cambiar la semántica de un endpoint.



¿Cuándo NO versionar?

Cuando el cambio es aditivo y no rompe nada: agregar un campo opcional nuevo a la respuesta, agregar un endpoint nuevo, mejorar el rendimiento sin cambiar el contrato. Los clientes que no usan el campo nuevo simplemente lo ignoran.



ESTRATEGIAS DE VERSIONAMIENTO

VERSIONAMIENTO EN LA URL

La más común y visible. La versión vive en la URL. Fácil de probar, fácil de enrutar, fácil de documentar en Swagger. La que vas a usar tú.

- `GET /api/v1/users`
- `GET /api/v2/users`

VERSIONAMIENTO EN EL HEADER

Más "pura" en REST — la URL no cambia. Pero es más difícil de probar, no aparece bien en Swagger y confunde a equipos pequeños.

- `GET /api/users`
- Header: `Accept: application/vnd.myapp.v2+json`



URL VERSIONING EN SPRING BOOT — PASO A PASO

```
// Paso 1 — Desde el inicio, pon /api/v1 en el @RequestMapping
@RestController
@RequestMapping("/api/v1/platos")
@Tag(name = "Platos v1", description = "Gestión de la carta")
public class PlatoController {
    // tus endpoints normales...
}

// Paso 2 — Cuando necesites una versión 2 que rompa compatibilidad,
//          crea un controller nuevo. El v1 sigue vivo para clientes viejos.
@RestController
@RequestMapping("/api/v2/platos")
@Tag(name = "Platos v2", description = "Carta con soporte de alérgenos")
public class PlatoV2Controller {
    // nueva estructura de respuesta con PlatoV2ResponseDTO
}
```

```
// Paso 3 — Si muchos endpoints cambian, usa un prefijo global en application.yml
server:
    servlet:
        context-path: /api    ← todos los endpoints quedan bajo /api

// 0 en cada controller:
@RequestMapping("/api/v1/platos")    ← manualmente, más explícito
@RequestMapping("/api/v2/platos")    ← la nueva versión
```



VERIFICACIÓN – ¿TIENES V1 EN TODOS TUS ENDPOINTS?

- Busca en tu proyecto todos los @RequestMapping y @GetMapping, @PostMapping, etc.
- ¿Todos empiezan con /api/v1/? Si no → corrégelos ahora antes de que empecemos a consumirlos
- En Swagger UI verifica que todas las rutas muestren /api/v1/ como prefijo
- Si tienes /platos sin prefijo → ese endpoint va a ser difícil de versionar después





REGLA DE ORO

Usa `/api/v1/` desde el primer endpoint que escribas, aunque hoy solo tengas una versión. Cambiarlo después con el frontend ya conectado es mucho más costoso que ponerlo desde el principio. Una URL con versión es un compromiso con el cliente: "este contrato no cambia sin previo aviso".

¿Agregar un campo nuevo a la respuesta requiere una nueva versión de la API?

No, si el campo es opcional y aditivo. Los clientes que no lo conocen lo ignoran. Solo necesitas nueva versión cuando eliminas o cambias algo que el cliente ya usa.

¿Por qué es mejor el URL versioning que el header versioning para este proyecto?

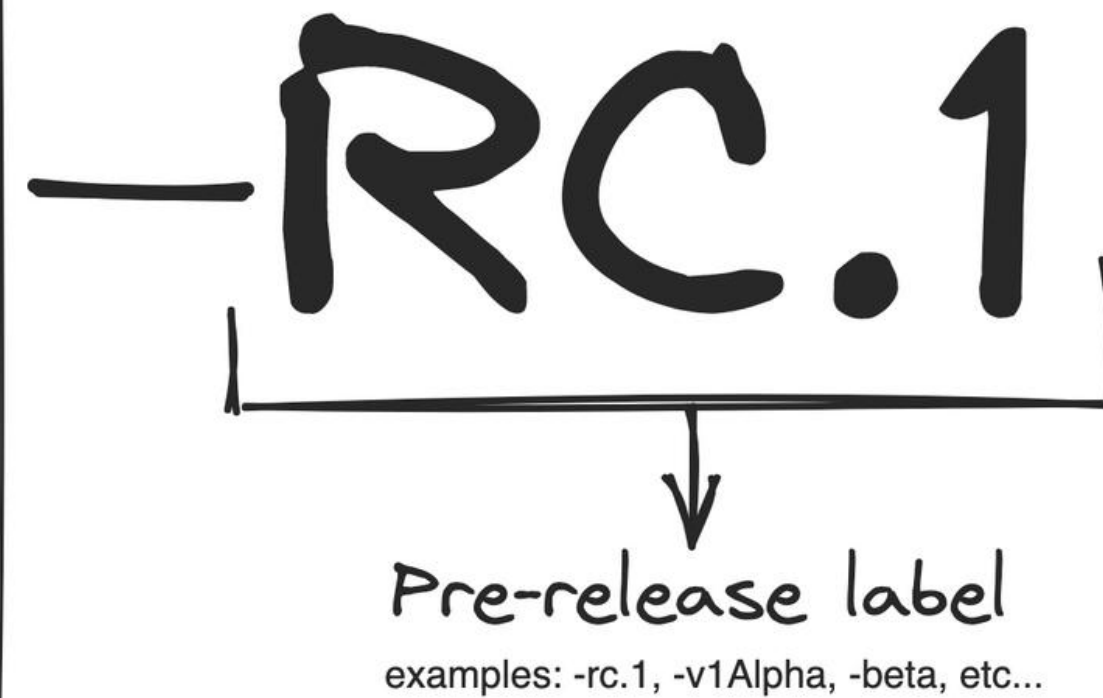
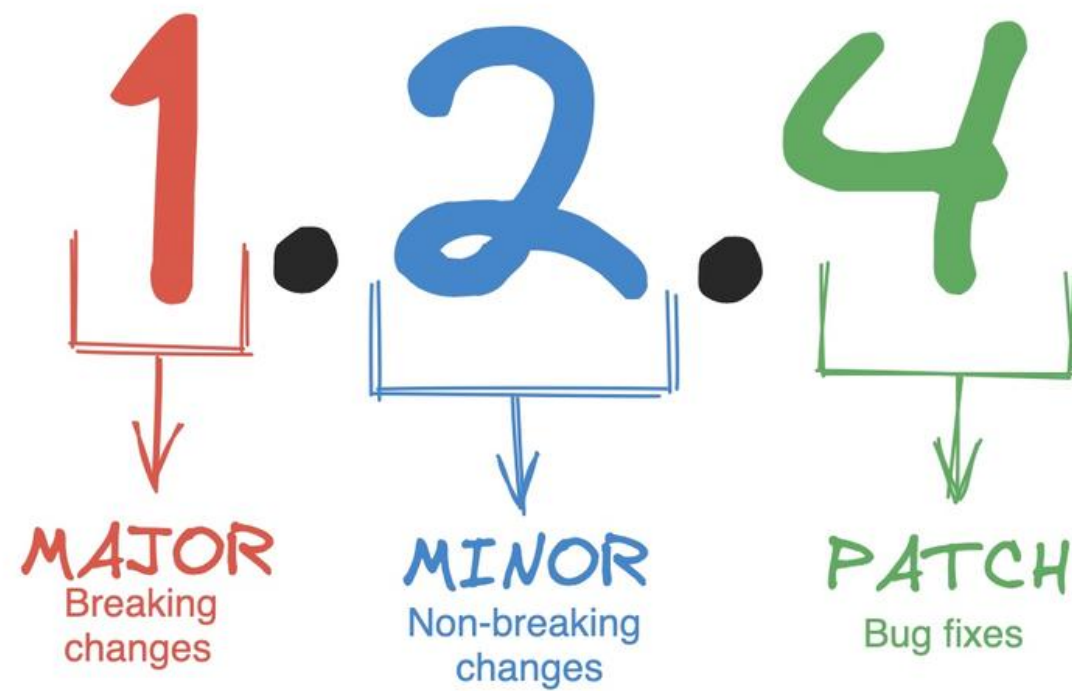
Porque es visible en Swagger, fácil de probar en Postman, fácil de enrutar y entendible para cualquier miembro del equipo sin conocer los headers HTTP.



VERSIONAMIENTO DE RAMAS

Release version

Optional pre-release suffix





PRUEBAS - CONFIAR PERO VERIFICA

Al probar el Service no quieres que las pruebas dependan de la BD
ni del Controller. Un mock es un doble que tú controlas.



¿POR QUÉ MOCKS?

Al probar el Service, no quieres que las pruebas dependan del Controller ni de la BD. UnMockes un doble de una dependencia que tú controlas — puedes decirle "cuando te llamen con este input, devuelve este output" sin que exista la implementación real.

Stub

Devuelve datos predefinidos. No verifica si fue llamado. Solo "responde" lo que le programas. Útil para simular el Repository con datos fijos.

Mock

Devuelve datos predefinidos Y verifica si fue llamado (cuántas veces, con qué argumentos). Con Mockito:

```
verify(mock, times(1)).metodo() .
```

Spy

Envuelve un objeto real — llama el método real pero también verifica las interacciones. Útil cuando quieres comportamiento real pero monitoreado.



DOSW COMPANY

IMPLEMENTACION

1

```
// test/service/PlatoServiceTest.java
@ExtendWith(MockitoExtension.class) // ← activa Mockito en JUnit 5
class PlatoServiceTest {

    // — Mocks de las dependencias —————
    @Mock
    private PlatoMapperIn mapperIn; // ← Mockito crea un doble vacío

    @Mock
    private PlatoMapperOut mapperOut;

    // — El servicio real, construido con los mocks inyectados —————
    @InjectMocks
    private PlatoServiceImpl platoService; // ← Mockito inyecta los @Mock en el constructor

    // — Fixtures - datos de prueba reutilizables —————
    private PlatoRequestDTO requestDto;
    private PlatoResponseDTO responseDto;
    private Plato platoEntidad;

    @BeforeEach // se ejecuta antes de CADA test
    void setUp() {
        requestDto = new PlatoRequestDTO("Tacos", 15000.0, "PLATO_FUERTE", "Deliciosos tacos");
        platoEntidad = new Plato(null, "Tacos", 15000.0, "PLATO_FUERTE", "Deliciosos tacos", true);
        responseDto = new PlatoResponseDTO(1L, "Tacos", 15000.0, "PLATO_FUERTE", true);
    }
}
```



DOSW COMPANY

IMPLEMENTACION

2

```
// — ESCENARIO EXITOSO: crear un plato nuevo —————
@Test
@DisplayName("✅ Crear plato válido — debe retornar ResponseDTO con id asignado")
void crear_platoValido_debeRetornarResponseDTO() {

    // GIVEN — comportamiento de los mocks
    when(mapperIn.toDomain(requestDto)).thenReturn(platoEntidad);
    when(mapperOut.toDTO(any(Plato.class))).thenReturn(responseDto);

    // WHEN — ejecutar el método bajo prueba
    PlatoResponseDTO resultado = platoService.crear(requestDto);

    // THEN — verificar el resultado
    assertNotNull(resultado);
    assertEquals("Tacos", resultado.nombre());
    assertEquals(15000.0, resultado.precio());
    verify(mapperIn, times(1)).toDomain(requestDto); // ← verifica que el mapper fue llamado
    verify(mapperOut, times(1)).toDTO(any(Plato.class));
}
```

3

```
// — ESCENARIO DE ERROR: nombre duplicado → debe lanzar excepción —————
@Test
@DisplayName("❌ Crear plato con nombre duplicado — debe lanzar PlatoYaExisteException")
void crear_nombreDuplicado_debeLanzarExcepcion() {

    // GIVEN — primero creamos el plato (sin mocks para este paso simple)
    when(mapperIn.toDomain(requestDto)).thenReturn(platoEntidad);
    when(mapperOut.toDTO(any(Plato.class))).thenReturn(responseDto);
    platoService.crear(requestDto); // primer creado — queda en el map

    // WHEN & THEN — el segundo intento debe lanzar la excepción
    PlatoYaExisteException ex = assertThrows(
        PlatoYaExisteException.class,
        () → platoService.crear(requestDto) // ← mismo nombre → debe fallar
    );

    assertTrue(ex.getMessage().contains("Tacos"));
}
```

Deben tenerse en cuenta las demas pruebas necesarias de los demas escenarios: Plato no encontrado, obtener platos disponibles, cambiar estado de pedido en estado incorrecto, etc – Pueden apoyarse de la i.a para la generacion de las pruebas



ESCENARIOS QUE SIEMPRE DEBES CUBRIR EN EL SERVICE:

- ✓ Caso exitoso (happy path) — el flujo normal funciona
- ✓ Recurso no encontrado — lanza la excepción correcta
- ✓ Duplicado / conflicto — lanza la excepción correcta
 - ✓ Estado inválido — lanza la excepción correcta
- ✓ Lista vacía — devuelve lista vacía, no null ni excepción

Ejecuten mvn test. ¿Cuántas pruebas pasaron? ¿Hay alguna en rojo?

¿Cuál es la diferencia entre @Mock y @InjectMocks?

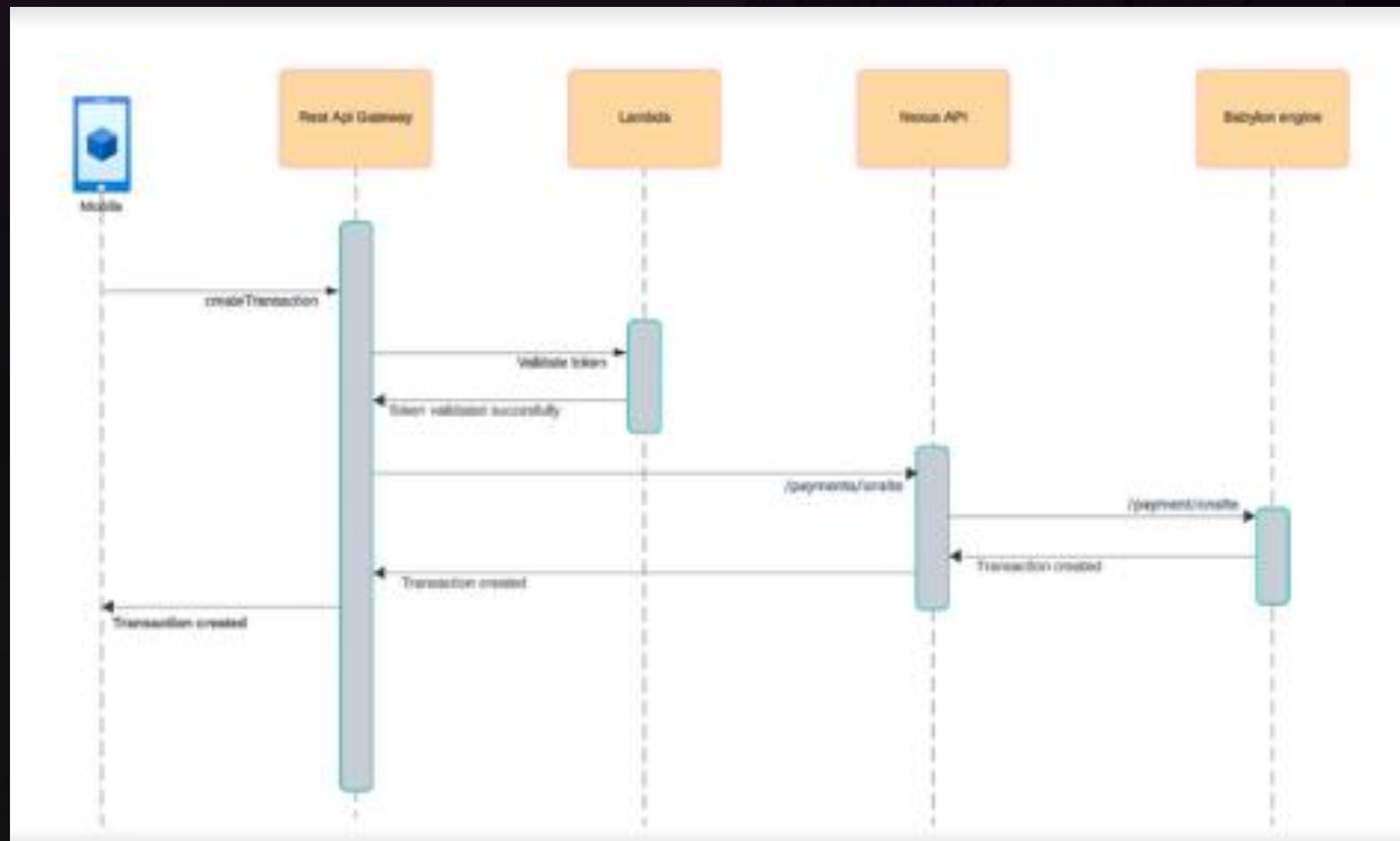
@Mock crea un doble vacío de una dependencia. @InjectMocks crea la clase real bajo prueba e inyecta los @Mock en su constructor. El SUT siempre es @InjectMocks.

¿Qué verifica verify(mock, times(1)).metodo()?



MODULO DE DIAGRAMACION #4

DIAGRAMAS DE SECUENCIA



•¿Qué muestra un diagrama de secuencia?

La interacción entre los componentes a lo largo del tiempo. No la estructura (eso es el diagrama de componentes), sino el flujo concreto de mensajes para un caso de uso específico.

Cada línea vertical es un componente (lifeline), cada flecha es un mensaje.

•Esencialmente, los diagramas de secuencia ofrecen una visualización cronológica de las interacciones de objetos, brindando perspectivas sobre el comportamiento dinámico y flujo de un sistema.

•Son especialmente útiles para visualizar y comprender las interacciones entre diferentes partes de un sistema a lo largo del tiempo



DIAGRAMAS DE SECUENCIA

Lifeline (Línea de vida)

Línea vertical discontinua con el nombre del componente arriba. Representa un participante activo en la interacción (Controller, Service, Repository, etc.).

Mensajes síncronos / asíncronos

Flecha sólida = síncrono (espera respuesta). Flecha discontinua = respuesta de retorno. Flecha abierta = asíncrono (no espera).

Activation bar

Rectángulo delgado sobre la lifeline — indica que ese componente está activo procesando una petición.

Fragmentos (alt, opt, loop)

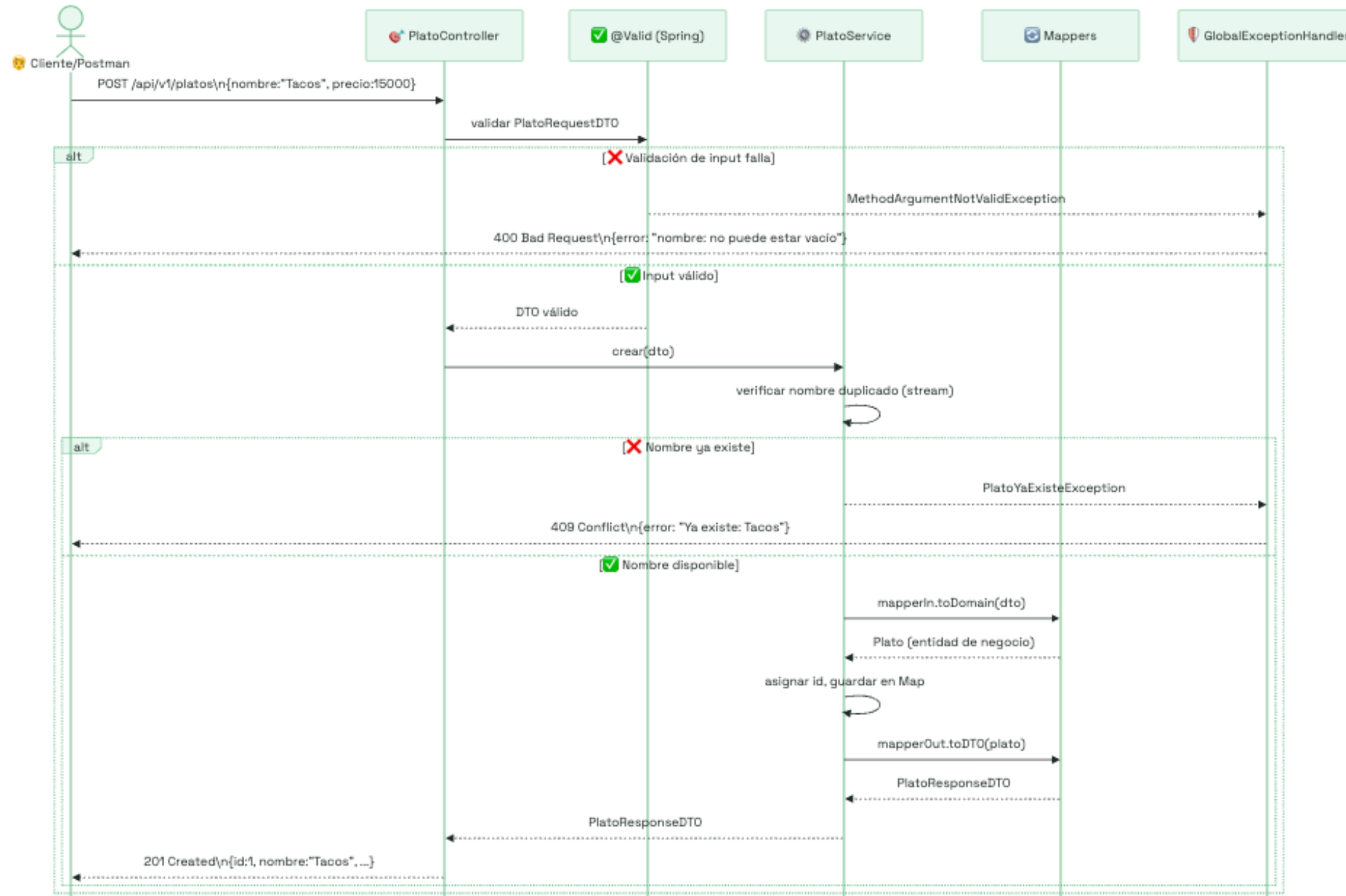
`alt` para condiciones (como if/else). `opt` para acciones opcionales. `loop` para repeticiones. Se etiquetan en la esquina superior izquierda de la caja.



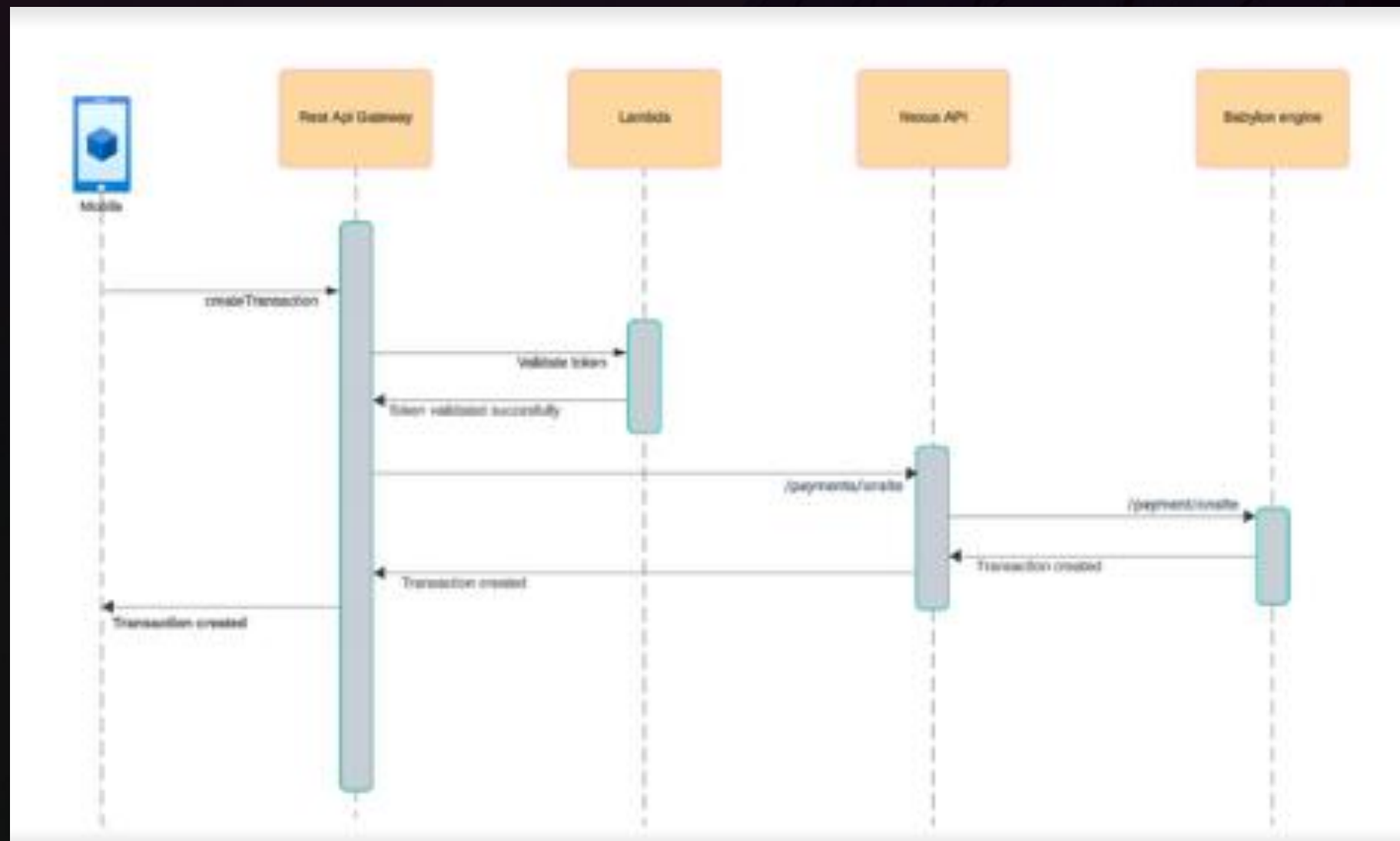
COMO NOS QUEDARIA EL DIAGRAMA DE SECUENCIA PARA

Crear un Plato – TANTO EN HAPPY PATH COMO ERROR

♦ DIAGRAMA DE SECUENCIA — CREAR UN PLATO EN EL RESTAURANTE (HAPPY PATH + ERROR)



DIAGRAMAS DE SECUENCIA



- Líneas de vida que se ven como líneas verticales discontinuas con el nombre del componente en la parte superior
- Mensajes que marcan las interacciones entre líneas de vida representadas como flechas, los mensajes síncronos se muestran como flechas negras, los asíncronos como flechas blancas y los mensajes de respuesta son flechas con líneas discontinuas
- Barras de activación que muestran la duración de la actividad, siendo los rectángulos delgados sobre la línea de vida
- Fragmentos que encapsulan comportamientos complicados o condicionales como bucles o alternativas o acciones concurrentes (alt, opt, loop, par), suelen aparecer estas etiquetas en la parte superior izquierda de la caja.



QUE VA EN CADA
DIAGRAMA?



ERROR FRECUENTE

Poner DTOs, anotaciones Spring o entidades JPA en el diagrama de clases. O meter el dominio de negocio en el diagrama de componentes. Cada diagrama habla de una capa distinta del sistema.

DIAGRAMA DE CLASES

Muestra las **entidades puras del dominio** con sus atributos, métodos y relaciones. Si una clase tiene `@Service`, `@Entity` o `@RestController`, **no va aquí**.

✓ **Va en este diagrama:**

Plato · Mesa · Pedido · ItemPedido · Cuenta · Reserva · RegistroVehiculo · EstadoPedido (enum) · EstadoMesa (enum)

✗ **No va aquí:**

PlatoController · PlatoService · PlatoRequestDTO · PlatoResponseDTO · PlatoMapper

DIAGRAMA DE COMPONENTES ESPECÍFICO

Muestra la **arquitectura interna del sistema** en detalle: componentes, sus interfaces y cómo se conectan entre capas. Símbolos UML formales: componente (rectángulo con pestañitas), lollipop (o—), socket (⌋).

✓ **Va en este diagrama:**

PlatoController · IPlatoService · PlatoServiceImpl · IPlatoMapper · PlatoMapperImpl · IPlatoRepository (semana 9)

✗ **No va aquí:**

Atributos de los objetos · Código de los métodos · DTOs internos

DIAGRAMA DE COMPONENTES GENERAL

Vista **macro del sistema completo**: el Core Backend, el Frontend Web, los sistemas externos (Maps, IA, pagos) y cómo se conectan entre sí. No entra al detalle de las capas internas.

✓ **Va en este diagrama:**

Sistema Core (Spring Boot) · Sistema Web (React) · Base de Datos · APIs externas (Maps, pagos, IA) · Actores principales

✗ **No va aquí:**

Controllers individuales · Services · Capas internas del backend



TAREA

1. Complementen sus DTOS (request y response) con @Valid para las demas funcionalidades
2. Creen las interfaces service y sus implementaciones
3. Creen los mappers necesarios para las demas implementaciones
4. Agreguen @Slf4j, streams y excepciones a sus metodos implementados y a los restantes
5. Generen las correctas validaciones de input y de negocio
6. Manejen sus excepciones y su Exception Handler – Pueden hacer 1 general o 1 por agrupacion de dominio
7. Realicen las pruebas de sus servicios – Apoyo de I.A y usen JUNIT y mockito
8. Apliquen las Reglas de negocio propias de tu concepto de Restaurante
9. Recuerden arreglar diagramas con los cambios que hicimos hoy

DENTRO DE HOY EN 8 la API debe quedar terminada – sin persistencia



NOS VEMOS EN

Proyecto terminado

El man que no hizo nada:



LA SESION DE TRABAJO DEL PROYECTO